

像素场景中的水面 雨雪与夜空

Mushreb 视频研究与雨后灯港制作参考

整理日期 2026 年 10 月 7 日

先做能正确涨落的岸线，再做折射、焦散和倒影。这是这组材料对《雨后灯港》最有用的制作顺序。雪天也应拆成空中的雪花和表面的积雪；两者共用天气状态，但不必同步出现或消失。

本文围绕 Mushreb 的五期 2D 游戏开发日志，补充一条 Godot 渲染测试线索。动态水位、雪天、夜空三章依据你提供的完整转录重新整理；雨天、视角旋转、ECS 渲染目前仅核实到原站标题和简介。工程建议与示例参数另行标明，不冒充作者原话或实测结论。

已核实的目录与材料边界

用户给出的账号 387103369，在 B 站公开视频列表中显示为 Mushreb。列表读取时显示 47 个投稿，但当前文本只展开一部分；本文不是全部 47 个视频的字幕合集。[1]

视频	时长	本次可用材料
DEV LOG.01 等距视角旋转	1 分 22 秒	原站标题与简介，未取得字幕
DEV LOG.02 雨天效果	2 分 07 秒	原站标题与简介，未取得字幕
DEV LOG.03 动态水位河流	4 分 12 秒	用户完整转录，原站简介交叉核实
DEV LOG.04 夜空倒影	1 分 55 秒	用户完整转录，原站简介交叉核实
DEV LOG.05 雪天效果	4 分 01 秒	用户完整转录，原站目录交叉核实
Godot C# ECS 渲染测试	未核实	原站标题与简介，未取得字幕

这次没有成功取得任何原站字幕文件，也没有逐帧观看视频。直接视频页及公开元数据接口在当前读取工具中未能稳定打开；可读的列表和搜索索引用于核对目录。没有据此推断视频“没有字幕”，也没有编造章节时间戳。

阅读顺序：做水边场景看第 2 至 3 页；做天气看第 4 至 5 页；做夜景看第 6 页；Canvas2D 接入和验收看第 8 页。末页列出已验证链接和仍待补齐的材料。

动态水位先从素材结构做起

依据：你提供的动态水位转录；视频标题和简介对应 DEV LOG.03。简介自述环境为 Godot 4.6.3 与 C#。[4] 以下素材流程来自转录，具体数据约定为本文建议。

地面和水分开画

先做完全没有水的河床与岩壁，再准备控制水的图。原作者使用双网格瓦片：从可重复拼接的底面开始，以辅助线约束边缘，复制到其他位置后调整纹理。岩壁轮廓可以不规则，但连接位置必须保持一致。碎石用于打破重复感，不能掩盖接缝问题。

每完成一批轮廓就放进引擎拼接。至少检查直岸、内角、外角、窄水道和连续重复的长岸线。若等所有纹理都画完才发现边界不连贯，修补会同时影响底色、阴影和水位辅助图。

用一条基准岸线推导各层水位

转录的关键操作是：先画一条水位线，复制后向下移动 1 像素，再逐层微调。逐层修改的目的，是保留同一块岩壁的地形关系。低水位时，要补出露出水面的石头周围的高光。

把这些水位范围按灰度或其他单调数值叠成控制图，得到手绘的分层深度信息。岸边高光另存一张分层图。它们都必须与无水底图保持相同尺寸、原点和瓦片布局，导出时不要各自裁切透明边缘。

素材	内容	最容易出错的地方
无水底图	河床、岩壁、石头、固有阴影	把旧水色或水线画死在底图
分层控制图	每个位置在哪一档水位被淹没	灰度顺序反了，水位上升反而退水
岸边高光图	每一档水位对应的接触边线	与水体阈值不一致，出现悬空白边
动态噪声	小幅水线运动及折射扰动	周期太短，整条河同步呼吸

美术验收比加特效更早

先把水画成一种纯色，只用滑杆扫过所有水位。逐档检查水是否连通、石头是否正确露出、拼接处有没有一格高一格低。控制图不必模拟真实三维深度；它首先是一份可预测的美术数据。只有需要水量、浮力或通行判断时，才另外定义它和游戏世界高度之间的关系。

对《雨后灯港》的建议：第一版先用有限的离散水位让台阶、桩脚和岸线可读。不要为了“连续”把所有像素边缘平滑化。此处是实现建议，转录没有指定总层数。

水位渲染和水量逻辑分开管理

依据：动态水位转录。下述函数关系是解释用的原创伪代码，不是视频源码；它们采用“高度越大，越不易被淹没”的统一约定。

用同一份水位驱动水和高光

设 height 是控制图记录的局部地形高度，level 是当前水位。waterMask = step(height, level) 表示水位达到该位置时显示水。编码方向若相反，应反转控制图或比较关系，而不是到处修正符号。

localDepth = max(level - height, 0) 可作为透明度和折射的驱动值，但只有控制图与 level 使用同一尺度时才有意义。若控制图只是离散档位标签，应先映射到有序高度，不能把色号当物理深度。

岸边高光取当前水位对应的层。一个常见错误是把所有低于当前水位的高光都叠上去，留下多条白色阶梯。先确认高光图是“某层的线”还是“累计淹没区”，再决定选层或求差分。

按可读性逐层加水的质感

1. 先叠水色。浅水保留更多河床信息，深水减弱河床可见度。程序变量若叫 alpha，应记住 alpha 大意味着更不透明；转录中的“透明度高”指看得更透，不应直接照抄成更大的 alpha。
2. 再加入水线微动。噪声让岸边高光轻微上下变化，但应受水体范围约束。否则低水位时白线会漂到干燥岩壁上。
3. 焦散只出现在水下。转录中的“胶伞”结合上下文按“焦散”理解，即水底的移动亮纹；这一术语校正没有通过原视频画面复核。亮纹需要与水深或水量联动，不能铺满整个画面。
4. 最后做折射。用噪声偏移水下底图的采样位置，并让偏移幅度随深度变化。采样结果限制在合适的水下区域，避免把岸上灯柱拉进河床。

示例起点：先测试 0 至 1 个逻辑像素的岸线运动和 0 至 2 个逻辑像素的折射。数值是本文为小尺寸像素场景提出的调试范围，尚未在《雨后灯港》中测量。以实际岸线是否抖动、河床是否还看得懂为准。

水量不是水位滑杆的同义词

转录最后提出可按格计算水量，但没有给出流体算法。要做相邻格交换，应单独保存每格水量、底部高度和容量，并按统一规则计算流量；渲染只读取结果。等量加水在不同形状的格子里，不一定造成相同幅度的水位上涨。

只做氛围原型时，一个全局水位足够。做闸门、潮汐解谜或洪水传播时，再验证总水量、相邻格交换上限和时间步长。不要为了展示涨水效果，先引入尚无玩法需求的流体模拟。

雪花粒子需要形状变化和寿命变化

依据：你提供的雪天完整转录，对应 DEV LOG.05。原站页面索引将作者环境列为 Godot 4.7.0 与 C#；这是视频简介自述，不是本文对该版本的兼容性测试。[6]

图集的九不是切分参数的九

作者用 9×9 像素画布画九种雪花，每种占 3×3 像素。图集有 3 列、3 行，所以 CanvasItemMaterial 的 H Frames 与 V Frames 都应填 3，而不是 9。Godot 官方说明确认，粒子图集需要在 CanvasItemMaterial 中启用粒子动画并配置行列数。[8]

给 GPUParticles2D 指定图集后，设置两种材料：process_material 使用 ParticleProcessMaterial 来控制运动；CanvasItem 的 material 使用 CanvasItemMaterial 来解释图集。只设置前者，不能完成图集切分。

将出生时的动画偏移随机化，使每片雪花从九格中选择形状。若希望雪花在飞行中保持同一形状，动画速度应为零；这一点是本文的实现补充，转录主要演示了随机 offset。

参数	转录中的设定	使用时应注意
图集尺寸	9×9 像素	九种形状，每格 3×3 像素
H Frames 与 V Frames	各 3	含义是列数与行数
重力	清零	主要用初速度与方向塑造飘落
发射形状	Box	范围按镜头和漂移距离调整
初始角度	0 至 360 度	极小雪花旋转可能改变像素轮廓
初始速度	50 至 150	作者示例值，不能脱离场景尺度照搬
spread	调小	未给出精确数值
lifetime 与随机性	适当增加	未给出精确秒数和比例

出生和消失不要抢眼

用 alpha 寿命曲线让雪花从透明进入可见，再在结束前淡出。转录明确说明曲线值 1 为完全显示、0 为隐藏。这样能减弱粒子从空气中突然冒出或骤然消失的感觉。

发射框应比可见区域更宽，给风向漂移留余地。不要只在屏幕上边缘一条细线上生成雪花，否则一开始没有近处雪花，风向一变还会露出空区。Canvas2D 版本可直接维护雪花对象池，以出生时固定的 spriteIndex、速度、相位和寿命绘制，不需要模拟 Godot 的全部节点结构。

测试时分别把背景换成纯黑、浅灰和灯光区域。雪花应在暗处清晰，在灯光里仍保留层次；增加粒子数量不能代替这些检查。

积雪用稳定蒙版在两套素材之间过渡

依据：雪天转录。作者先用噪声与 step 控制积雪范围，再像素化 UV；更完整的做法，是用蒙版在原贴图和覆盖雪的贴图之间混合。

为什么只叠白色会损失体积

屋顶、台阶和箱子的原图已经有阴影和朝向。直接把蒙版内的像素刷成同一片白色，会抹掉这些信息。为重要物体画雪版本，可以把积雪放到屋顶、横梁和石头顶部，并保留雪下的厚度、背光面和投影。噪声只决定露出多少雪版本，不能替代这份美术判断。

制作顺序可以是：原始素材 → 同尺寸雪素材 → 固定噪声 → 阈值蒙版 → 原图与雪图混合。snowColor = mix(baseColor, coveredColor, snowMask) 是解释这一关系的伪代码，不是作者提供的完整 shader。

固定噪声 只移动阈值

积雪慢慢增厚时，建议固定噪声种子和空间坐标，只改变覆盖阈值。若每帧重新随机，雪斑会不断跳动，看起来像电视噪点。噪声 UV 先按逻辑像素量化，再采样，可防止积雪边缘出现与原图不一致的细软纹理。

本文约定 cover 从 0 到 1 表示由无雪到全覆盖。可以让阈值随 cover 降低，但要显式处理 cover = 0 和 cover = 1：分别返回全黑、全白蒙版，避免 step 的边界相等规则留下孤立点。这里是工程补充，视频没有给出阈值边界代码。

把对象表面的积雪资格单独控制也很有用。屋顶可积雪，屋檐下、门洞和水面应有各自规则。建筑雪图与原图使用相同锚点，不能因为换图让窗框、台阶或阴影跳动。

下雪强度和积雪量要分别记录

飘雪强度决定当前有多少雪花；积雪量记录地表已经留下多少雪。停雪后保留积雪，再缓慢消融，天气变化会更有连续性。若按下“停止下雪”就同时把地面清空，两个系统的关系会显得机械。

对《雨后灯港》的建议：先做屋顶和码头上表面两类雪素材，保留水面不积雪的遮罩。测试覆盖 0%、25%、50%、75%、100% 五个状态，确认屋顶边缘不会被噪声吞掉、路面仍可辨认、不同物体的雪斑不会按同一节奏机械重复。这些比例是本文验收采样点，不是作者设置。

夜空先合成 再交给水面采样

依据：你提供的夜空完整转录，对应 DEV LOG.04。作者解释，俯视游戏中直接看不到天空，因此通过水中倒影展示星空、云和极光。简介自述使用 Godot 4.7.0 与 C#。[5]

星星的随机性来自固定网格

作者先切分世界空间，对每一格做哈希，再结合格内坐标决定该格是否出现星星及星星大小，随后加入单向运动和闪烁。这里最重要的是随机结果与格子绑定；不应每帧调用无种子的随机数重新决定所有星点。

本文建议把星星的“是否存在”“亮度相位”“微小尺寸差异”分开取值。星点可以很少，闪烁也可以错开。若每颗星同一时刻变亮，画面会像一张反复调透明度的贴纸。相机运动时，还要检查星点是跟随世界、天空还是屏幕；选择可以不同，但不能无意混用。

云位于星空与极光之间

转录中的极光从动态噪声开始，沿垂直方向逐层取样，给不同层叠加不同颜色的渐变。原转录写作“REMARCHING”，本文按语境理解为逐步采样累积；没有据此扩展成完整三维体积光算法。

作者特别指出，演示的是倒影，因此向下步进；若做直接抬头看到的天空，应向上。实际接入时必须先决定合成目标到底是正常朝向的天空，还是已经翻转过的倒影。否则水面再翻一次，就会把方向弄反。

云层放在星空与极光之间，再调整色调和遮挡关系。三层都应受同一夜间状态管理。极光不必在所有地图出现；转录提到它服务于特定地域的夜景，而不是通用的每夜天气。

一个倒影来源保持各元素一致

作者把所有倒影内容画进同一个 Viewport，再由水面 shader 采样，并指向雨天一集作为这一部分的说明。Godot 官方资料支持把 SubViewport 的结果作为 ViewportTexture 使用。[10] 这不意味着本文已经复原了雨天一集的全部方案。

在 Canvas2D 中，可将夜空和需要倒映的灯光、建筑画到离屏 canvas，再在水区内按条带或网格采样。各部分使用同一相位和时间源，避免天空静止、灯影突然跳到另一帧。画倒影时不要把已经包含倒影的主画布再次作为输入，防止递归残影。

水中应还能辨认出星点、云带与极光的层次。先关闭波纹确认合成和方向，再开启小幅扰动；过大的偏移会把所有内容搅成同一块颜色。对于《雨后灯港》，灯影是近处视觉锚点，夜空提供远处氛围，亮度应分别控制。

雨天与渲染测试目前能确认什么

本页仅依据原站标题、简介以及夜空转录中的交叉指向。没有雨天、视角旋转、ECS 三条视频的可核对完整字幕，因此不将推测补写成作者教程。

雨天效果

DEV LOG.02 的主题是 2D 游戏中的雨天效果，时长 2 分 07 秒，简介自述 Godot 4.5.1 与 C#。[3] 夜空转录明确说，倒影统一渲染到 Viewport 后由水面 shader 采样的做法，可参考这一集。因此，把它作为倒影管线的后续重点材料是有依据的。

当前不能确认作者怎样生成雨滴、怎样处理屋檐遮雨、落地水花或积水增长，也不能从标题推断其实现。本文给《雨后灯港》的独立建议是把空中雨线、表面水花、积水遮罩分开绘制；它们共享风向和雨强，但使用不同遮罩。此建议不归入视频内容。

如果后续获得雨天字幕，应优先补四件事：倒影来源包含哪些对象；Viewport 如何对齐主相机；水面遮罩如何生成；雨滴与表面互动是否只是视觉效果。这些答案会直接影响已有水面方案，优先级高于继续堆叠粒子。

等距视角旋转

DEV LOG.01 的简介说，作者将此前渲染系统改造成多层 Tilemap，并讨论当前旋转视角方案及伪影问题。[2] 这只能支持“多层地图与旋转是该视频主题”，不能支持某一种矩阵、插值或遮挡算法就是作者方案。

《雨后灯港》若仍采用固定镜头，暂不为这个主题重构渲染器。以后确实需要旋转时，再检验墙体背面、人物朝向、Y 排序、像素对齐和遮挡。模糊或抗锯齿是否可接受，应由最终像素风格决定，不能因为简介提到模糊就直接全局开启。

Godot C# ECS 渲染测试

原站简介称测试自动打包 2000 多个不同纹理，并实现实时 YSort 和视锥剔除。[7] 这是作者自述的测试内容；没有获得硬件、分辨率、实体数量、帧耗时分布或源码，不能把它当成本项目可达到的性能承诺。

能借鉴的研究方向是减少纹理切换、只绘制可见对象、让遮挡顺序可预测。但 Canvas2D 原型不必立即改成 ECS。先测量每帧耗时，确认瓶颈在绘制数量、离屏合成还是逐像素读写，再决定优化目标。

目录中还有哪些值得继续看

同一账号列表还出现 Ballex² 重做记录中的造景工具、编辑器更新前瞻、基础控制器与摄像机，以及 Scene 系列概念视频。[1] 当前只验证标题存在，未取得这些视频的正文或可靠直链。它们应保留为后续选题，不应混入已读教程的技术结论。

雨后灯港的接入顺序与验收

本页是基于三篇转录提出的工程方案，没有检查当前项目源码，也没有声称下列功能已经实现。Canvas2D 与 Godot 可以共用素材约定和效果分层，但不是逐行移植同一份 shader。

先统一三类坐标

美术使用逻辑像素，游戏状态使用世界坐标，显示再统一缩放。噪声、水位图、倒影图和对象锚点都应明确属于哪一类坐标。最终放大时使用最近邻采样，避免清晰的素材被输出层重新抹软。Godot 官方对 nearest 的定义是只读取最近的像素。[9]

建议的绘制顺序为：干燥地形和水下底图 → 河床焦散与受限折射 → 水色和倒影 → 当前水位岸线 → 前景遮挡对象 → 天气粒子和局部水花。该顺序需要按场景调整；桥梁、栈道和人物遮挡必须单独测试，不能仅靠一种全局 Y 排序解决全部对象。

每次只启用一个新效果

次序	做什么	通过的条件
1	纯色水位和岸线	所有档位连续，拼接无裂缝
2	透明度与折射	浅水见底，岸上物体不被拉入水下
3	统一倒影画布	朝向正确，移动镜头不发生独立漂移
4	雪花与积雪	雪花形状稳定，停雪后积雪可保留
5	多种天气切换	风向和强度变化连续，没有突变清屏
6	性能测量	最复杂场景下记录帧耗时及设备条件

在 Canvas2D 里，静态控制图可以预计算；不要未经测量就在每帧对整张画布执行逐像素读取与重写。水面波纹可先用有限条带采样验证美术效果，再决定是否需要 WebGL。这个取舍需要真实设备测量，本文未给出性能保证。

若使用 Godot 的屏幕读取 shader，要注意它默认不会为每一个后续对象重复抓取屏幕。需要新内容时，应明确安排 BackBufferCopy 或独立 Viewport；区域复制还要覆盖被采样的位置，避免读取未复制区域。[11]

保留可复现的测试场景

准备一张包含内外转角、浅滩、深水、石头、桥柱、屋檐和一盏灯的小地图。固定随机种子和相机位置，保存无效果、水位最低、水位最高、雨夜、停雪后五组对照图。每次调材质时先用这张地图比较，再看大场景。

最值得盯住的不是特效数量，而是这些具体问题：水位变化是否真正改变可见河床；高光是否贴着接触边；雪是否落在合理表面；灯与夜空的倒影是否属于同一个世界。把这些问题逐项解决，场景会比单纯增加噪声和光晕更扎实。

来源与仍待补齐的材料

以下链接于 2026 年 10 月 7 日通过公开视频列表、原站索引或官方文档核实。原站部分直链在当前工具中未能打开；保留的是已由搜索结果验证的地址，不表示已观看或下载视频。

[1] Mushreb 公开视频列表，账号 387103369。用于核实账号和可见投稿目录。

<https://www.bilibili.com/list/387103369/?bvid=BV1s6rDBpEvo&oid=115911302322320>

[2] 2D 等距视角的视角旋转 DEV LOG.01。2025 年 12 月 4 日。

<https://www.bilibili.com/video/BV1x32vBREVZ/>

[3] 2D 游戏中的雨天效果 DEV LOG.02。2026 年 1 月 18 日。

<https://www.bilibili.com/video/BV1s6rDBpEvo/>

[4] 2D 游戏中的动态水位河流效果 DEV LOG.03。2026 年 6 月 15 日。

<https://www.bilibili.com/video/BV1KyjN6iEzq/>

[5] 2D 游戏中的夜空倒影效果 DEV LOG.04。2026 年 6 月 28 日。

<https://www.bilibili.com/video/BV143TM6pEDq/>

[6] 2D 游戏中的雪天效果 DEV LOG.05。原站合集和账号列表核实标题与 4 分 01 秒时长；雪天的独立 BV 链接本次未核实。请从[1]的同名视频进入。原站收藏页索引显示发布时间为 2026 年 9 月 9 日，其默认播放项目会变化，因此不把收藏页当作稳定视频直链。

[7] Godot+C# ECS 渲染测试 包含实时 Y 排序与视锥剔除。2025 年 11 月 28 日。

<https://www.bilibili.com/video/BV1YRSEBwEgA/>

[8] Godot 官方 GPUParticles2D 参考。用于核对粒子材料与图集设置。

https://docs.godotengine.org/en/stable/classes/class_gpuparticles2d.html

[9] Godot 官方 Viewport 参考。用于核对最近邻采样含义。

https://docs.godotengine.org/en/stable/classes/class_viewport.html

[10] Godot 官方 SubViewport 纹理教程。仅用于核对渲染到纹理这一机制。

https://docs.godotengine.org/en/stable/tutorials/shaders/using_viewport_as_texture.html

[11] Godot 4.6 官方屏幕读取 shader 说明。用于核对屏幕副本及 BackBufferCopy 限制。

https://docs.godotengine.org/en/4.6/tutorials/shaders/screen-reading_shaders.html

正文的详细步骤主要来自你在 2026 年 10 月 7 日提供的水位、雪天和夜空三篇完整转录。明显的语音识别误词已按上下文整理，如粒子、Aseprite、Godot、雪和 alpha；焦散与逐步采样的术语解释已在相关章节注明。本文未复制或重新分发完整字幕，也未取得视频源码。

待补齐：雨天原字幕与倒影具体对齐方法；视角旋转与 ECS 的完整讲解；雪天独立 BV 直链；剩余投稿的逐项筛选。获得这些材料后，应更新相应章节及其证据说明，保留当前内容中哪些是作者做法、哪些是本文建议的区别。